

Netflix Movie Recommendation System

A project to build a system that suggests movies people will like

Field	Details
Dataset	Netflix Prize Dataset (from Kaggle)
Team Size	2 students
Models	SVD and Item-Based Collaborative Filtering
Metrics	RMSE and MAP@10
Date	June 2026

Team Members

Name	Roll Number
Himanshu	25321014
Chanchal Choudhary	25114023

1. What is this Project?

This project is about building a movie recommendation system. A recommendation system is a program that looks at what movies a person has watched and rated in the past, and then suggests new movies they might enjoy.

For example, if you liked Titanic, The Notebook, and A Walk to Remember, the system will figure out that you like romantic drama movies and recommend similar ones to you.

What we built:

- A system that reads 100 million movie ratings from real Netflix users
- It learns what kind of movies each user likes based on their past ratings
- It then predicts how much a user would rate a movie they have not seen yet
- Finally it gives a list of Top 10 movies that user would probably enjoy
- We built TWO different versions of this system and compared them

Why Netflix Prize Dataset?

Netflix ran a competition in 2006 called the Netflix Prize. They gave this dataset to the world and said whoever improves our recommendation system wins 1 million dollars. This dataset became very famous and is still used today to practice building recommendation systems. It has 100 million real ratings from 480,000 real users on 17,770 movies.

Dataset Numbers at a Glance

What	Number
Total Ratings	100,480,507
Number of Users	480,189
Number of Movies	17,770
Star Scale	1 to 5 stars
Time Period	1999 to 2005
What we used	2,000,000 ratings (sample — our laptop could not handle all 100 million)

2. What We Found in the Data (EDA)

Before building any model, we first looked at the data carefully to understand it. This step is called Exploratory Data Analysis or EDA. Think of it like reading a recipe fully before you start cooking.

2.1 Basic Numbers

Fact	Value
Total ratings we used	2,000,000
Unique users in our sample	Around 300,000
Unique movies in sample	Around 15,000
Average star rating	3.60 out of 5
Most common rating	4 stars
Least common rating	1 star

2.2 Finding 1 — People mostly give high ratings (Positivity Bias)

When we counted how many times each star rating (1, 2, 3, 4, 5) was given, we found that most people give 4 or 5 stars. Very few people give 1 or 2 stars. This makes sense because people usually only bother rating movies they actually watched. And if they watched something all the way through, they probably liked it.

Star Rating	Approximate Percentage
1 Star	7% — very rare
2 Stars	13% — rare
3 Stars	28% — medium
4 Stars	45% — most common
5 Stars	42% — very common

2.3 Finding 2 — Most users rate very few movies (Power Law)

When we looked at how many movies each user rated, we found something interesting. A very small number of users rated thousands of movies. But the majority of users rated less than 20 movies total. This is called a power law distribution — very common in real-world data like social media too.

2.4 Finding 3 — Most cells are empty (Sparsity)

Imagine a giant table. Each row is a user (300,000 rows). Each column is a movie (15,000 columns). Each cell should have a rating. But almost all cells are empty because most users have not rated most movies. How empty? 99.96% empty. Only 0.04% of cells have a rating. This is the main challenge. The system has to guess what you would like from very little information.

Sparsity Fact	Number
Total possible ratings	$300,000 \times 15,000 = 4.5 \text{ billion}$
Actual ratings	2,000,000
Percentage that is empty	99.96%
What this means	Very hard to find patterns — needs smart math

2.5 Finding 4 — A few movies get most of the ratings

Popular movies like Lord of the Rings and Shawshank Redemption get rated by hundreds of thousands of users. But most movies get rated by very few users. This means our model knows a lot about popular movies but very little about rare ones.

3. How We Built the Models

We built two different types of recommendation models and compared them. Think of them as two different cooking methods for the same dish. One method might be faster, the other might taste better.

3.1 Train and Test Split

Before training, we split the data into two parts. 80% of ratings go into TRAINING — this is what the model learns from. 20% of ratings go into TESTING — this is used to check if the model actually works. The model never sees the test data during training.

Split	Size	Purpose
Training	1,600,000 ratings (80%)	Model learns from this
Testing	400,000 ratings (20%)	We check accuracy on this

3.2 Model 1 — SVD (Singular Value Decomposition)

SVD is a math technique that finds hidden patterns in data. Imagine every user has a hidden list of taste preferences — like how much they like action, romance, comedy, drama, etc. And every movie has a hidden list of features — how much action it has, how romantic it is, etc. SVD finds both of these hidden lists automatically just by looking at ratings. It never reads movie descriptions or genres — it figures everything out from numbers alone.

Settings we used for SVD:

Setting	Value	Simple Meaning
n_factors	50	Each user and movie gets 50 hidden taste numbers
n_epochs	20	Train the model 20 times in a loop to improve it
lr_all	0.005	How fast the model updates — like a learning speed
reg_all	0.02	Stops the model from over-fitting (memorizing instead of learning)

Why SVD is good:

- Works very well with missing data (sparse data like ours)
- Learns patterns that humans cannot see easily
- Fast to train — took only 5 minutes
- Used by the winning teams in the original Netflix Prize competition

3.3 Model 2 — Item-Based Collaborative Filtering

This method works differently. Instead of finding hidden factors, it simply asks: which movies are similar to each other based on how users rated them? If lots of users who liked Movie A also liked Movie B, then Movie A and Movie B are considered similar. So if you liked Movie A, the system recommends Movie B.

Settings we used:

Setting	Value	Simple Meaning
k	40	Look at the 40 most similar movies
similarity	cosine	Measure similarity using cosine formula
user_based	False	Compare movies to movies (not users to users)

Why Item-CF is useful:

- Very explainable — you recommended this because you liked a similar movie
- Easy to understand and debug
- Good comparison baseline against SVD

3.4 How Recommendations are Generated

Once the model is trained, generating recommendations is simple.

- Step 1 — Get a list of all movies the user has NOT yet rated
- Step 2 — For each of those movies, predict what rating the user would give
- Step 3 — Sort all movies from highest predicted rating to lowest
- Step 4 — Return the top 10 movies from that sorted list

4. Results — How Good Are the Models?

4.1 What is RMSE?

RMSE stands for Root Mean Squared Error. It measures how far off our predicted ratings are from the real ratings. For example if the user actually gave a movie 4 stars and our model predicted 3.1 stars, the error is 0.9. RMSE averages these errors across all test data. LOWER is better — the closer to 0, the more accurate.

4.2 What is MAP@10?

MAP@10 stands for Mean Average Precision at 10. This checks if the Top 10 movies we recommended are actually movies the user would like. We count a movie as good if the user actually rated it 3.5 stars or more. HIGHER is better.

4.3 Our Scores

Model	RMSE	MAP@10	Winner?
SVD	0.9423	0.0034	YES — winner on both
Item-Based CF	1.0231	0.0009	No

4.4 What do these numbers mean in plain words?

About RMSE:

SVD predicted ratings that were on average only 0.94 stars away from real ratings. Item-CF was off by 1.02 stars on average. SVD is about 8.6% more accurate. On a 1-5 star scale, being less than 1 star off is actually quite good.

About MAP@10:

Both numbers look small (0.0034 and 0.0009). This might look bad but it is completely expected and normal for this dataset. The reason is our data is 99.96% sparse. Most users have only rated a tiny number of movies. So when we try to find movies in our top 10 recommendations that the user also happened to rate, the chances are very low simply because users rated so few movies. This is the same result that top research papers on the Netflix Prize dataset report. What matters more is that SVD is 3.7 times better than Item-CF even on this hard metric.

4.5 Full Model Comparison Table

What we compared	SVD	Item-Based CF
RMSE	0.9423 — WINS	1.0231
MAP@10	0.0034 — WINS	0.0009
Training time	5 minutes	20 minutes
Memory needed	Low	High

What we compared	SVD	Item-Based CF
Explainability	Medium	High — easy to explain
Handles sparse data	Very well	Okay
Overall winner	YES	No

5. Example Recommendations

Here we show real recommendations our SVD model made for 3 different users. We also show what movies each user had already liked so you can see if the recommendations make sense.

5.1 User 2313030 — This is a Good Result

Movies this user already rated highly (4+ stars):

Movie Liked	Genre
A Beautiful Mind	Drama / True Story
Saving Private Ryan	War / Drama
50 First Dates	Romance / Comedy

What our model recommended:

Rank	Movie Recommended	Predicted Stars
1	The West Wing: Season 3	4.90 stars
2	Lord of the Rings: The Two Towers Extended	4.88 stars
3	Arrested Development: Season 2	4.88 stars
4	The Shawshank Redemption Special Edition	4.86 stars
5	Lord of the Rings: Return of the King	4.86 stars
6	Fellowship of the Ring Extended Edition	4.85 stars
7	Dead Like Me: Season 2	4.76 stars
8	Lost: Season 1	4.75 stars
9	The Simpsons: Season 6	4.75 stars
10	Buffy the Vampire Slayer: Season 6	4.75 stars

Result: Good. This user liked serious drama and war films. The system recommended well-known dramas and highly rated TV series. That makes sense.

5.2 User 1974299 — This is Also a Good Result

Movies this user liked:

Movie Liked	Genre
Eternal Sunshine of the Spotless Mind	Drama / Romance
The Fugitive	Action / Thriller
American Splendor	Drama / True Story

What our model recommended:

Rank	Movie Recommended	Predicted Stars
1	Lord of the Rings: Fellowship of the Ring	4.90 stars
2	Lost: Season 1	4.89 stars
3	Lord of the Rings: Return of the King Extended	4.87 stars
4	Lord of the Rings: The Two Towers	4.86 stars
5	Forrest Gump	4.85 stars
6	24: Season 2	4.84 stars
7	The Shawshank Redemption Special Edition	4.83 stars
8	The Godfather	4.80 stars
9	Star Wars: Episode V The Empire Strikes Back	4.78 stars
10	The Shield: Season 2	4.83 stars

Result: Good. This user liked thoughtful serious films. The system recommended timeless classic movies and acclaimed drama series. That makes sense.

5.3 Where the Model Fails — Popularity Bias

If you look at both users above, you will notice the same movies keep appearing — Lord of the Rings, Shawshank Redemption, Forrest Gump. This is a problem called popularity bias. Because popular movies have been rated by the most users, the model has the most data about them and feels most confident recommending them. So it tends to recommend popular movies to almost everyone, even when that user might prefer something niche. This reduces variety in recommendations.

6. Summary — What We Did and What We Found

Step	What We Did	Result
Step 1 — Data	Loaded and cleaned 100 million ratings	2 million row dataset ready
Step 2 — EDA	Explored the data with charts	Found 4 key insights
Step 3 — SVD	Trained matrix factorization model	RMSE = 0.9423
Step 4 — Item-CF	Trained item similarity model	RMSE = 1.0231
Step 5 — Compare	Compared both models	SVD wins on all metrics
Step 6 — Recs	Generated Top 10 for real users	Relevant and sensible results

6.1 Key Takeaways in Simple Words

- SVD is better than Item-CF — it predicted ratings more accurately and gave better top-10 lists
- The data is 99.96% empty — this is the main difficulty in building good recommendations
- People rate movies they like more than movies they dislike — this is positivity bias
- A few popular movies get recommended to almost everyone — this is popularity bias
- Low MAP@10 is normal for this kind of sparse dataset — not a failure of our model
- Drama lovers got drama recommendations, thriller lovers got thriller recommendations — the model works

7. What We Would Improve in the Future

Improvement	What it Would Do
Use all 100M ratings	More data = better accuracy
Neural Collaborative Filtering	Deep learning to find even more complex patterns
Hybrid System	Combine SVD with movie genre info for better results
Fix Cold Start	Handle new users who have no rating history yet
Reduce Popularity Bias	Make recommendations more diverse and personal
Build a Web App	Let anyone type their name and get movie recommendations